

Introduction to Machine Learning Safety

Exercise Sheet 3

Submitted By: Sanjana Munikrishnappa

Exercise 3.1 – Computational Graphs (optional)

The function is: $f(x, y, z) = (x \cdot y)\sqrt{z} / \exp(x)$

Computational graph nodes:

$$a = x \cdot y$$

$$b = \sqrt{z}$$

$$c = a \cdot b = xy\sqrt{z}$$

$$d = \exp(x)$$

$$f = c / d$$

Derivatives computable from this graph:

Using backpropagation through this graph, we can compute the partial derivatives $\partial f / \partial x$, $\partial f / \partial y$, and $\partial f / \partial z$. For example, $\partial f / \partial y$ flows simply through the multiplication node a . The derivative $\partial f / \partial x$ is more complex because x contributes to both the numerator (through a) and the denominator (through $\exp(x)$), so the chain rule applies to both paths and the gradients are summed. The graph makes explicit which intermediate values must be stored during the forward pass so they can be reused in the backward pass.

Exercise 3.2 – Backpropagation (optional)

Backpropagation is an algorithm for efficiently computing the gradients of a loss function with respect to all parameters in a neural network.

During the **forward pass**, activations are computed layer by layer and stored. During the **backward pass**, the chain rule of calculus is applied in reverse order through the computational graph: the gradient of the loss is propagated from the output back through each layer, multiplying by local Jacobians at each node. This avoids redundant computation each intermediate gradient is computed exactly once making it far more efficient than computing derivatives from scratch for each parameter. The resulting gradients are then used by an optimiser (e.g. SGD or Adam) to update the network weights.

Exercise 3.3 – Gradient Descent (optional)

Gradient descent minimises a loss function $L(\theta)$ by iteratively moving the parameters in the direction of steepest descent.

Update rule:

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_{\theta} L(\theta_t)$$

where η is the learning rate.

Inverted sign: If the sign were inverted ($\theta_{t+1} = \theta_t + \eta \cdot \nabla_{\theta} L$), the update would move in the direction of steepest *ascent*, maximising the loss rather than minimising it. The parameters would diverge and the model would fail to learn.

Exercise 3.4 – Dataset Exploration

3.4.1 – Image counts

Split	Images
Training	7,200
Validation	3,600
Test	3,600

3.4.2 – Class distribution (training set)

Label	Present	Absent	Positive rate
Pedestrian	1,718	5,482	23.9%
Traffic light	5,276	1,924	73.3%
Vehicle	5,458	1,742	75.8%

The classes are **not balanced**. The pedestrian label is notably imbalanced — only ~1 in 4 images contains a pedestrian. Traffic light and vehicle are present in roughly three-quarters of images, so they are moderately imbalanced in the positive direction.

3.4.3 – Visual patterns

Images with pedestrians typically show figures near kerbs or crossings; absent images are often clear road scenes. Traffic-light-present images show urban intersections; absent images tend to be open road stretches. Vehicle-present images are almost ubiquitous, reflecting the dense traffic setting of CARLA Town02. All images are taken from a front-facing camera at constant height and angle, under clear daytime conditions.

Exercise 3.5 – Training Three Binary Classifiers

3.5.1 – Architecture and training setup

A **ResNet-18** pre-trained on ImageNet was fine-tuned for each of the three detection tasks. The final fully connected layer was replaced with a single output neuron (sigmoid applied at inference). Key hyperparameters:

Hyperparameter	Value
Optimiser	Adam
Learning rate	1×10^{-4}
Loss function	BCEWithLogitsLoss
Batch size	32
Epochs	10
Input size	224 × 224 (resize + centre crop)
Normalisation	ImageNet mean/std

3.5.2 – Loss curves and convergence

Model	Train loss (ep. 10)	Val loss (ep. 10)	Converged?
Pedestrian	0.0837	2.2651	No – overfitting

Model	Train loss (ep. 10)	Val loss (ep. 10)	Converged?
Traffic light	0.0038	0.0423	Yes
Vehicle	0.0277	0.1859	Yes

The traffic light and vehicle models converge well. The **pedestrian model does not converge**: training loss steadily decreases while validation loss increases monotonically — a clear sign of overfitting, likely driven by the severe class imbalance (only 24% positive examples) and the difficulty of detecting small pedestrians at varying distances.

3.5.3 – Why three separate models? (Safety perspective)

Separating the tasks offers several safety advantages:

- **Independent failure modes.** A single multi-label model could learn shared representations that cause correlated failures across all outputs simultaneously. Three separate models fail independently.
- **Targeted auditing.** Each classifier can be tested, monitored, and updated without affecting the others. Safety claims can be made per-hazard.
- **Simplified uncertainty estimation.** A dedicated binary classifier produces a single confidence score per hazard that is easier to threshold and interpret than a vector of correlated multi-label outputs.
- **Asymmetric criticality.** The three targets have different safety implications (e.g. missing a pedestrian is more critical than missing a traffic light). Separate models allow task-specific calibration and threshold tuning.

Exercise 3.6 – Evaluation

3.6.1 – Test-set metrics

Model	Accuracy	Precision	Recall	F1
Pedestrian	0.6125	0.2619	0.5368	0.3521
Traffic light	0.9417	0.9419	0.9791	0.9602
Vehicle	0.8486	0.9900	0.8063	0.8888

3.6.2 – Worst-performing model and hypothesis

The **pedestrian detector** performs worst across all metrics. Likely causes:

- **Class imbalance:** only 23.9% of training images contain a pedestrian, incentivising the model to default to 'absent'.
- **Appearance variability:** pedestrians vary greatly in size, clothing, and pose, making generalisation harder.
- **Overfitting:** training loss drops below 0.1 while validation loss exceeds 2.0 — the model memorises the scarce positive training examples.

3.6.3 – Precision vs. recall from a safety perspective

For safety-critical detection in autonomous driving, **recall matters more than precision** for all three models. The reasoning is asymmetric: a **false negative** (missing a real hazard) can cause a collision, whereas a **false positive** (unnecessary caution) only reduces efficiency or comfort.

Model	Why recall is critical	Recall achieved
Pedestrian	Missing a pedestrian could cause a fatal collision.	0.537 -- dangerously low
Traffic light	Failing to detect a red light could cause the vehicle to run it.	0.979 — excellent
Vehicle	Missing a vehicle ahead could lead to a rear-end collision.	0.806 — acceptable

Exercise 3.7 – ODD Gap Analysis

3.7.1 – Conditions covered by the training data

Inspecting the simulation config (*config.yaml*) and file structure, the training set covers exactly one operating condition:

Dimension	Training condition
Weather	Clear/sunny (precipitation=0, fog_density=2)
Lighting	Daytime (sun_altitude_angle=45°)
Map	Town02 (single urban map)
Camera	Fixed front-facing, constant height and angle
Traffic density	~25 vehicles, ~10 pedestrians (fixed seed=12345)

3.7.2 – ODD dimensions not covered by training data

ODD Dimension	In training data?
Fog / reduced visibility	No
Night-time / artificial lighting	No
Alternative map / road topology	No
Dusk / dawn / varying sun angle	No
Camera variations (noise, dirt, angle)	No
Rain / wet roads	No
Traffic density variations	No

3.7.3 – Safety implications

For ODD conditions not covered by training data, **no performance guarantees can be made**. The cross-condition recall results confirm this starkly:

Test split	Pedestrian recall	Traffic light recall	Vehicle recall
test (in-distribution)	0.537	0.979	0.806
test-night	0.501	0.470	0.814
test-fog	0.074	0.181	0.442
test-town-01	0.520	0.617	0.722

Performance collapses severely under fog. Traffic light recall drops from 0.979 to 0.181 — virtually useless. Vehicle recall halves. This means the system cannot safely operate in fog, at night, or on an

unfamiliar map without retraining or explicit ODD restrictions.

3.7.4 – Identified gaps to revisit in Sheets 4 and 5

- **Foggy weather** — complete performance collapse; the ODD must exclude fog or the models must be retrained on fog data.
 - **Night-time lighting** — significant degradation in traffic light detection; lighting conditions outside daytime must be excluded from the ODD or addressed with augmentation/domain adaptation.
 - **Alternative map layouts** (Town01 and beyond) — moderate degradation; map-generalisation must be tested further.
 - **Camera variation** (angle, noise, lens dirt) — not tested at all; no claims can be made.
 - **Weather extremes** (rain, high wind, dust) — not present in training; excluded from any safety claim.
-